

Cyber Security Internship

Task 1: Basic Network Sniffer

Task number	Task 1
Task title	Basic Network Sniffer
Submitted by	Muhammad Asif
Internship ID	CA/DF1/245989
Company	Code Alpha
Domain	Cyber Security



Table of Contents

1. ABSTRACT.....	3
2. Introduction.....	3
3. Problem Statement	3
4. Project Objectives	4
5. Technologies and Tools Used	5
7. Core Features	7
7.1 Live Packet Capture.....	7
7.2 Capture Filters	7
8. Protocol Analysis.....	8
9. IP Address Analysis	8
10. TCP and UDP Connection Analysis	9
11. Payload Analysis.....	9
Security Observation.....	10
12. DNS Analysis.....	10
13. DHCP Analysis.....	11
14. Traffic Statistics	12
15. PCAP Export	13
16. Wireshark Compatibility	13
17. Session and Memory Handling	13
18. Error Handling and Validation	14
19. Testing	14
20. Sample Results	15
21. Security and Ethical Considerations	15
22. Limitations	16
23. Future Improvements	16
24. Conclusion	17
25. Author	17

1. ABSTRACT

This paper presents the design and implementation of a Basic Network Sniffer, a Python-based command-line tool for live network traffic capture and analysis. Developed as part of the CodeAlpha Cyber Security Internship, the application leverages the Scapy library to capture packets from a network interface and extract critical information for network security and troubleshooting. The tool identifies various protocols (TCP, UDP, ICMP, ARP), analyzes IP addresses and connections, examines packet payloads, and provides deep inspection of application-layer traffic such as DNS queries and DHCP messages. To enhance usability, it features configurable traffic filters, generates comprehensive traffic statistics, and offers the option to export captured data to PCAP format for further analysis in tools like Wireshark. The project effectively demonstrates fundamental concepts in network security and analysis, fulfilling the core internship requirements while providing a practical, extensible utility for understanding network communication and flow.

2. Introduction

Network traffic contains important information about how devices communicate with each other. Understanding this traffic is an essential part of network security, troubleshooting, monitoring, and security analysis.

As part of the CodeAlpha Cyber Security Internship, this project implements a Python-based network packet sniffer using the Scapy library.

The tool captures network packets from a selected network interface and analyzes them to extract useful information such as protocols, source and destination IP addresses, ports, payload information, DNS queries, DHCP messages, and network connections.

The project goes beyond basic packet capture by providing configurable traffic filters, payload analysis, traffic statistics, and optional PCAP export for further investigation in tools such as Wireshark.

The original CodeAlpha requirement is to capture network traffic, analyze packet structure and content, understand protocol/network flow, and display information including source/destination IPs, protocols, and payloads.

3. Problem Statement

Network communication occurs through thousands of packets containing different protocols and types of information. Manually examining this traffic is difficult without appropriate tools.

The objective of this project is therefore to develop a lightweight command-line network sniffer capable of:

- Capturing live network packets
- Identifying network protocols

- Extracting source and destination IP addresses
- Identifying TCP and UDP connections
- Analyzing packet payloads
- Inspecting DNS queries
- Inspecting DHCP messages
- Generating traffic statistics
- Filtering traffic according to protocol
- Exporting captured packets to PCAP format

The tool provides a practical way to observe and understand network communication from a cybersecurity perspective.

4. Project Objectives

The main objectives of the project are:

1. Develop a Python-based packet capture tool.
2. Capture live network traffic using Scapy.
3. Analyze the structure and contents of captured packets.
4. Identify commonly observed protocols.
5. Display source and destination IP addresses.
6. Identify TCP and UDP connections and ports.
7. Analyze available packet payloads.
8. Monitor DNS queries.
9. Analyze DHCP communication when available.
10. Provide configurable capture filters.
11. Generate useful network traffic statistics.
12. Allow captured traffic to be saved as a PCAP file.
13. Verify exported captures using Wireshark.

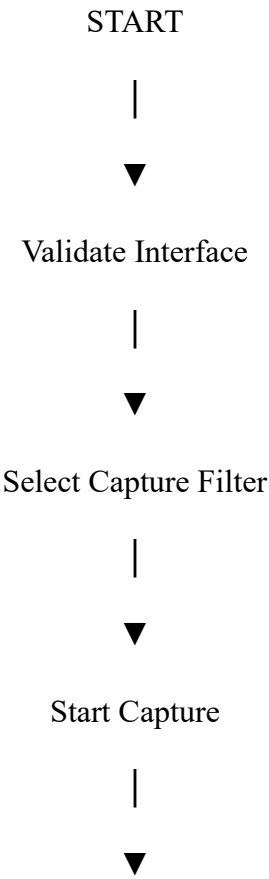
These objectives directly address the core requirements specified by CodeAlpha for the Basic Network Sniffer task.

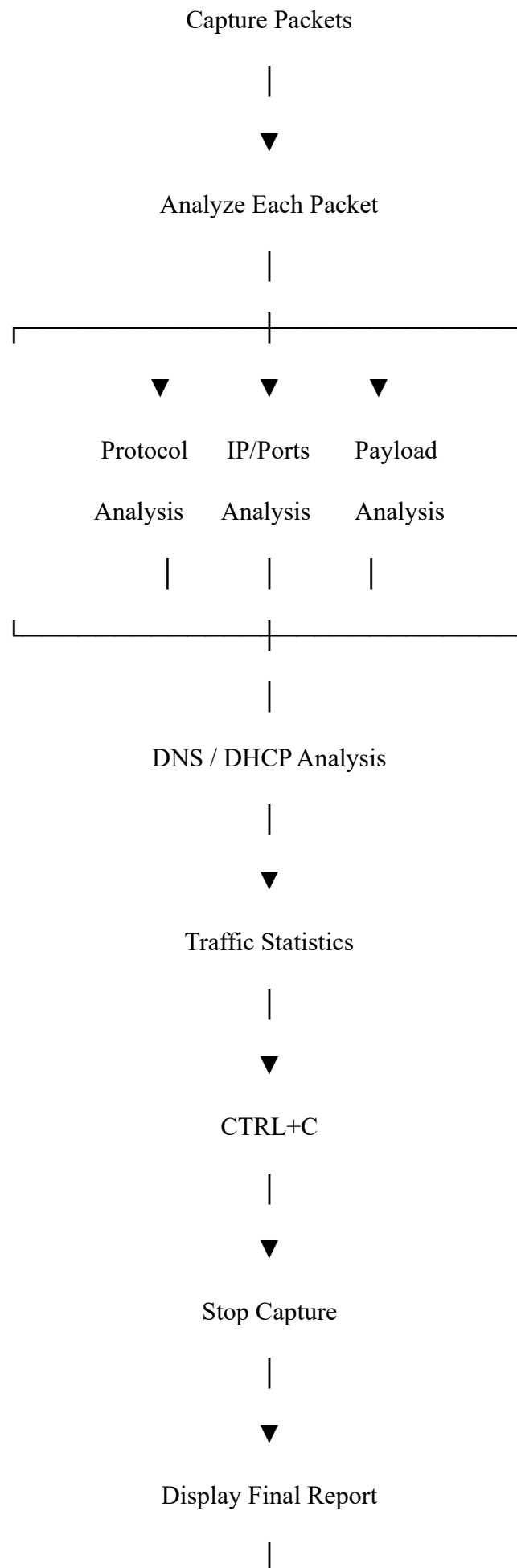
5. Technologies and Tools Used

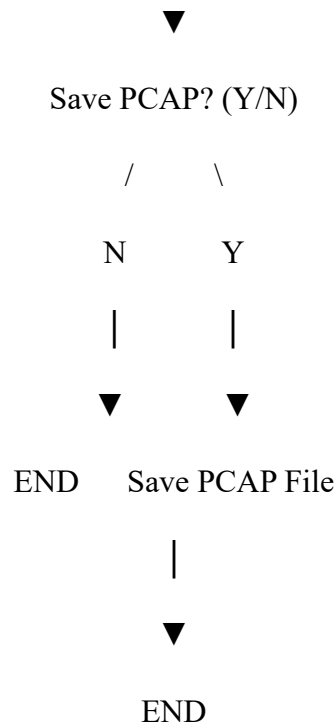
Technology / Tool	Purpose
Python	Main programming language
Scapy	Packet capture and packet analysis
Npcap	Windows packet-capture support
Wireshark	Verification and deeper PCAP analysis
VS Code	Development environment
PowerShell	Running and testing the application

6. System Architecture

The application follows this workflow:







7. Core Features

7.1 Live Packet Capture

The sniffer captures network traffic directly from the selected network interface using Scapy.

The application uses:

`sniff()`

with packet processing performed through a callback function.

Captured packets are analyzed immediately rather than being permanently stored in memory by Scapy's packet store.

7.2 Capture Filters

The application provides six filtering options:

1. All Traffic
2. TCP Only
3. UDP Only
4. DNS Only
5. HTTP/HTTPS

6. TCP + UDP

This allows the user to focus the capture on a specific type of traffic.

For example:

TCP Only

uses a TCP capture filter, while:

DNS Only

restricts the capture to DNS traffic.

8. Protocol Analysis

The application identifies several network and application protocols.

Network / Transport Protocols

- TCP
- UDP
- ICMP
- ARP

Application-Level Identification

- HTTP
- HTTPS
- DNS
- DHCP
- QUIC / HTTP3
- Unknown

The application determines protocols using Scapy packet layers and, where appropriate, TCP/UDP port numbers.

9. IP Address Analysis

For IPv4 packets, the application extracts:

Source IP

Destination IP

It maintains counters for both source and destination addresses.

The final report displays the most frequently observed addresses.

Example:

TOP SOURCE IP ADDRESSES

192.168.100.135	14383 packets
52.107.243.167	5765 packets
192.168.100.1	894 packets

This helps identify which hosts are generating or receiving significant amounts of traffic.

10. TCP and UDP Connection Analysis

For TCP and UDP packets, the application records:

- Source IP
- Destination IP
- Source port
- Destination port
- Protocol
- Packet count
- Total bytes

Example:

192.168.100.135:5908 → 52.107.243.167:443

TCP 3303 packets 4840268 bytes

This provides a concise view of the network conversations observed during a capture session.

11. Payload Analysis

One of the important components of the project is payload analysis.

For packets containing a Raw layer, the application extracts the payload and calculates:

- Payload size
- HEX representation
- ASCII representation

Example:

Application : HTTPS

Payload Size: 1412 bytes

HEX:

2e 75 32 d8 a4 a6 70 b9 f9 7e f5 e8 ...

ASCII:

.u2...p..~..@.{..~.u.6.....

Security Observation

HTTPS payloads commonly appear as apparently random binary data because the application data is encrypted.

Therefore, the sniffer can capture and measure encrypted payloads, but it cannot simply display the original HTTPS content.

This is an important practical observation from the project.

12. DNS Analysis

The sniffer analyzes DNS packets and records DNS queries.

It extracts:

- Client IP
- DNS server
- Requested domain
- Query type

Supported query types include:

- A
- AAAA

- CNAME
- MX
- NS
- PTR
- TXT
- SRV
- HTTPS/SVCB

Example captured queries included domains such as:

www.youtube.com

accounts.youtube.com

edge.microsoft.com

hif-dliq.deepseek.com

tpc.googlesyndication.com

The application also maintains counters to identify frequently requested domains.

13. DHCP Analysis

The application can analyze DHCP messages when DHCP traffic is present.

It identifies message types such as:

- DISCOVER
- OFFER
- REQUEST
- ACK
- NAK
- DECLINE
- RELEASE
- INFORM

It can also extract available information such as:

- Client MAC address
- Assigned/requested IP
- DHCP server IP
- Hostname

If no DHCP traffic occurs during a capture session, the application correctly reports:

DHCP Messages Detected : 0

No DHCP messages detected.

14. Traffic Statistics

After the capture is stopped, the application generates a complete traffic summary.

The report includes:

General Statistics

- Total packets
- Total bytes

Protocol Statistics

- TCP
- UDP
- ARP
- ICMP
- Other IP

Application Statistics

- HTTPS
- DNS
- DHCP
- QUIC/HTTP3
- HTTP
- Unknown

Network Statistics

- Top source IP addresses
- Top destination IP addresses
- Top connections

15. PCAP Export

The project includes optional PCAP export.

After the user stops the capture, the application asks:

Do you want to save this capture as a PCAP file? (Y/N)

If the user selects **Y**, a Windows Save dialog allows the user to select the destination and filename.

The capture is then saved in .pcap format.

This allows the captured traffic to be preserved for later investigation rather than existing only during the live capture session.

16. Wireshark Compatibility

The generated PCAP files were tested with Wireshark.

The purpose of this test was to verify that:

1. The capture is successfully written to disk.
2. The file can be opened by Wireshark.
3. Packets remain available after the Python program terminates.
4. Wireshark can independently analyze the captured traffic.

This makes the sniffer useful not only as a live analysis tool but also as a packet-capture acquisition utility.

17. Session and Memory Handling

During normal live capture, the program processes packets and maintains the required analysis information in memory.

The Scapy capture itself is configured with:

```
store=False
```

Therefore, the program does not maintain a complete Scapy packet list for the entire session.

Only selected information required for statistics, payload samples, DNS/DHCP details, and connection analysis is retained.

The PCAP becomes permanent only when the user explicitly chooses to save the capture.

This design reduces unnecessary memory usage during long capture sessions.

18. Error Handling and Validation

The final version includes validation and error handling for the capture process.

The application:

- Checks the network capture environment before starting.
- Handles invalid filter selections.
- Handles capture errors.
- Responds to CTRL+C.
- Signals the capture thread to stop.
- Waits for the capture thread to terminate.
- Reports errors without unnecessarily terminating the entire analysis process.
- Handles PCAP save cancellation.

This improves the reliability of the command-line application.

19. Testing

The project was tested using different capture scenarios.

Test	Result
Start packet capture	✓ Passed
TCP filtering	✓ Passed
UDP filtering	✓ Passed
DNS filtering	✓ Passed
HTTP/HTTPS filtering	✓ Passed
TCP + UDP filtering	✓ Passed

IP detection	✓ Passed
Connection tracking	✓ Passed
Payload analysis	✓ Passed
DNS analysis	✓ Passed
DHCP analysis	✓ Passed
CTRL+C termination	✓ Passed
PCAP generation	✓ Passed
Wireshark compatibility	✓ Passed
Interface/error validation	✓ Passed

20. Sample Results

During testing, a TCP-filtered capture produced:

Total Packets Captured : 23287

Total Bytes Captured : 19159383

TCP : 23287 packets (100.00%)

Payload analysis reported:

Packets containing payload : 13761

Total payload bytes : 17831055

Packets with payload : 59.09%

The capture also detected DNS queries and multiple HTTPS connections.

These results demonstrate that the application successfully captured and analyzed real network traffic.

21. Security and Ethical Considerations

A packet sniffer can expose sensitive network information. Therefore, it should only be used on networks and devices where the user has appropriate authorization.

This project was developed for educational cybersecurity and network-analysis purposes.

The tool should not be used to intercept or inspect traffic belonging to other users or networks without permission.

22. Limitations

The project has several technical limitations:

1. HTTPS application content cannot normally be read because it is encrypted.
2. The application primarily analyzes IPv4 traffic.
3. Protocol identification based on port numbers is not always definitive.
4. Some modern traffic may use encrypted DNS or encrypted application protocols.
5. Packet capture availability depends on the operating system and Npcap configuration.
6. Very long capture sessions can still generate significant processing and analysis data.
7. The tool is intended for educational and basic network-analysis purposes rather than replacing professional IDS/IPS systems.

23. Future Improvements

Possible future improvements include:

- IPv6 support
- More advanced protocol identification
- Improved encrypted traffic classification
- Real-time packet display
- Export to CSV/JSON
- Traffic graphs
- Packet search functionality
- Advanced connection tracking
- Alert generation for suspicious patterns
- Modular analysis architecture
- Optional graphical interface

The GUI is intentionally not part of the current implementation because the project is designed as a command-line cybersecurity utility.

24. Conclusion

The Basic Network Sniffer project successfully implements a Python-based network traffic capture and analysis tool using Scapy.

The application captures live packets, identifies protocols, analyzes IP addresses and network connections, examines available payloads, monitors DNS and DHCP traffic, generates traffic statistics, supports configurable capture filters, and provides optional PCAP export for further investigation.

The project fulfills the core requirements of CodeAlpha's Task 1 — Basic Network Sniffer, which requires a Python program capable of capturing and analyzing network traffic and displaying information such as source/destination IP addresses, protocols, and payloads.

The completed implementation also provides additional functionality such as DNS/DHCP analysis, filtering, PCAP export, interface validation, and Wireshark compatibility, making it a more complete practical network-analysis tool.

25. Author

Muhammad Asif

Cyber Security Intern — CodeAlpha

Project: Basic Network Sniffer